

CZ2006/CE2006 Software Engineering

Lecture 19: Control Flow Testing

Liu Yang

Black Box Testing Pros and Cons

▶ **Applicability**

- ▶ All levels of system development (Unit, Integration, System, Acceptance)

▶ **Cons**

- ▶ Cannot know how much of the implementation have been tested
- ▶ No notion of testing coverage like in white box testing

▶ **Pros**

- ▶ Directs the tester to a very small subset of test inputs that can highly likely find the bugs

White Box Testing Process

The most difficult steps

► The tester

1. Analyzes SUTs (System Under Test) implementation
 - **Control Flow Graph** or Data Flow Graph
2. Identify **execution paths** through the SUT
3. Chooses **inputs** to cause the SUT to **execute selected paths**
4. Determines expected outputs (oracle) for chosen inputs
5. Constructs tests with the chosen inputs (e.g., write JUnit program)

► The testing engine (e.g., JUnit)

1. Executes the tests
2. Compares actual outputs (log) with the expected outputs (oracle)
3. Determines whether the SUT (System Under Test) functions properly

White Box Testing Pros and Cons

▶ **Applicability**

- ▶ All levels of system development (Unit, Integration, System, Acceptance)

▶ **Pros**

- ▶ Ensure that every path through the SUT has been identified and tested

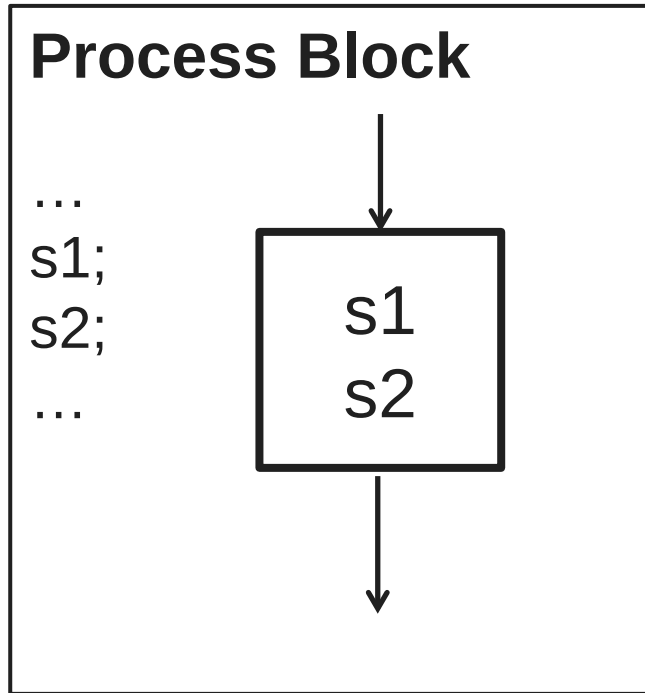
▶ **Cons**

- ▶ Testing all execution paths is generally infeasible
 - ▶ e.g., loop, #decision points
- ▶ May not detect data and arithmetic bugs
 - ▶ e.g., $a = a - 1$ // should be $a = a + 1$; or
 a/b // b cannot be zero
- ▶ Never find paths that are not implemented
 - ▶ e.g., paths in requirements are missing in implementation

Control Flow Graph (CFG)

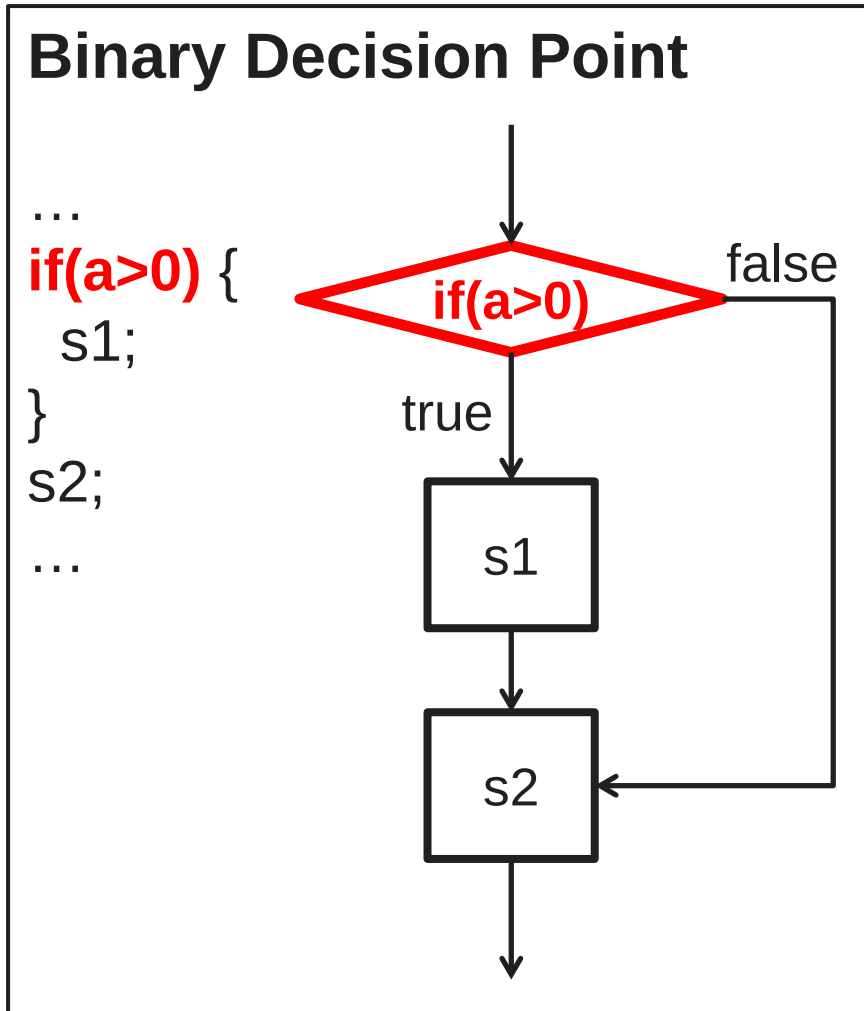
- ▶ A directed graph, consists of nodes and edges, where:
 - ▶ Each node represents a process block or a decision point
 - ▶ Each edge represents control flow (i.e., what happens after what)

Process Block



- ▶ A sequence of statements **execute sequentially**
- ▶ Does not contain if/while/for/switch/goto statements
- ▶ Contains any number of sequentially executed statements in the process block
- ▶ One control flow edge into the process block
- ▶ One control flow edge out of the process block

Binary Decision Point – if statement

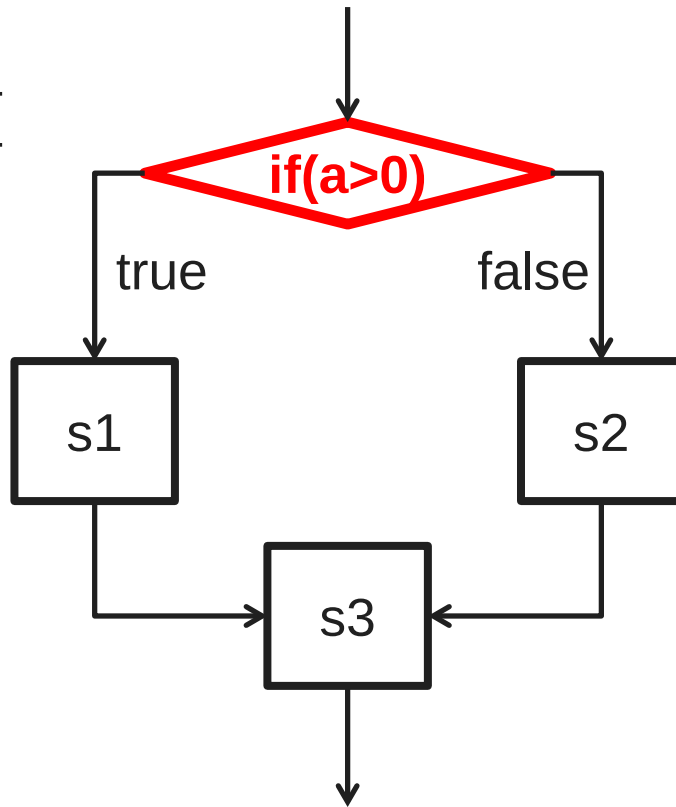


- ▶ Control flow can change at the decision point
- ▶ One control flow edge into the decision point
- ▶ **Two** control flow edges out of the decision point
 - ▶ True branch
 - ▶ False branch

Binary Decision Point – if-else statement

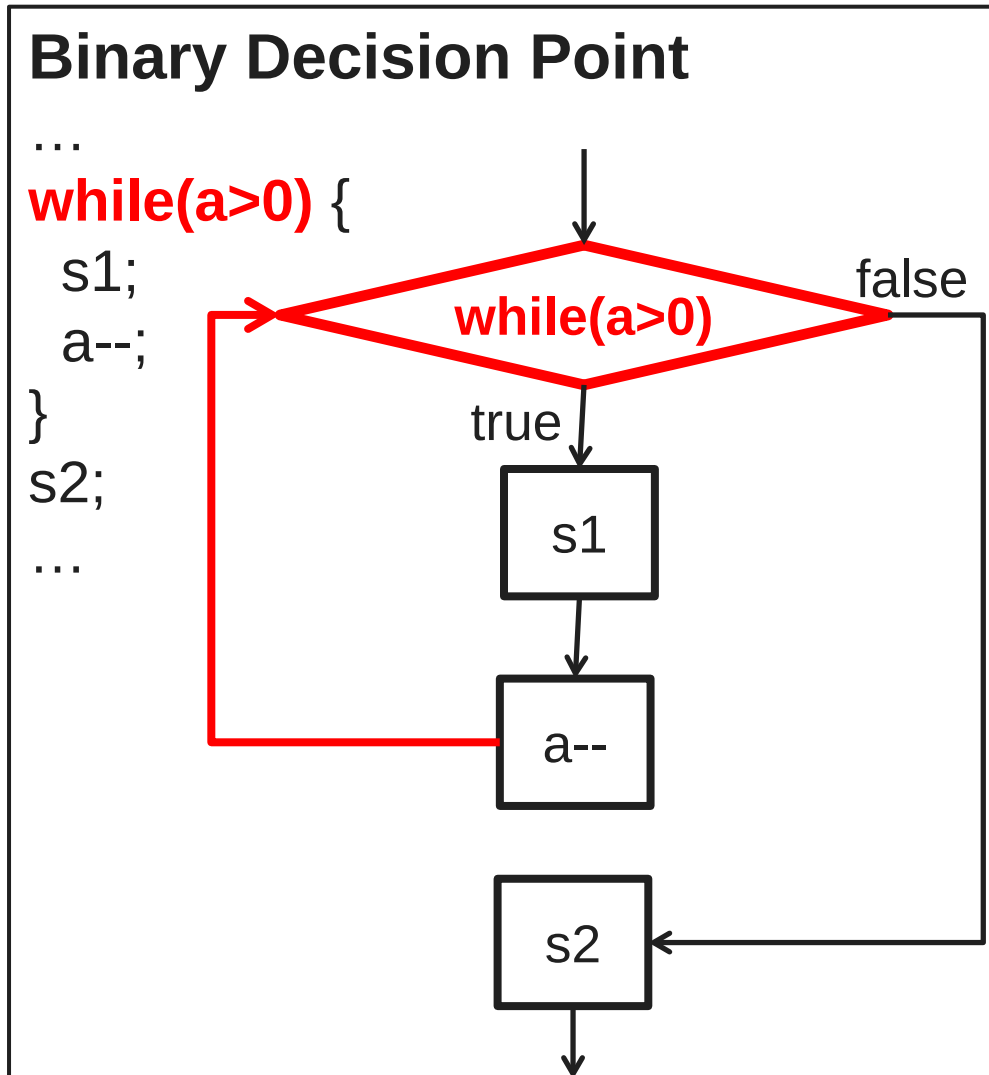
Binary Decision Point

```
...  
if(a>0) {  
  s1;  
}  
else {  
  s2;  
}  
s3;  
...
```



- ▶ Control flow can change at the decision point
- ▶ One control flow edge into the decision point
- ▶ **Two** control flow edges out of the decision point
 - ▶ True branch
 - ▶ False branch

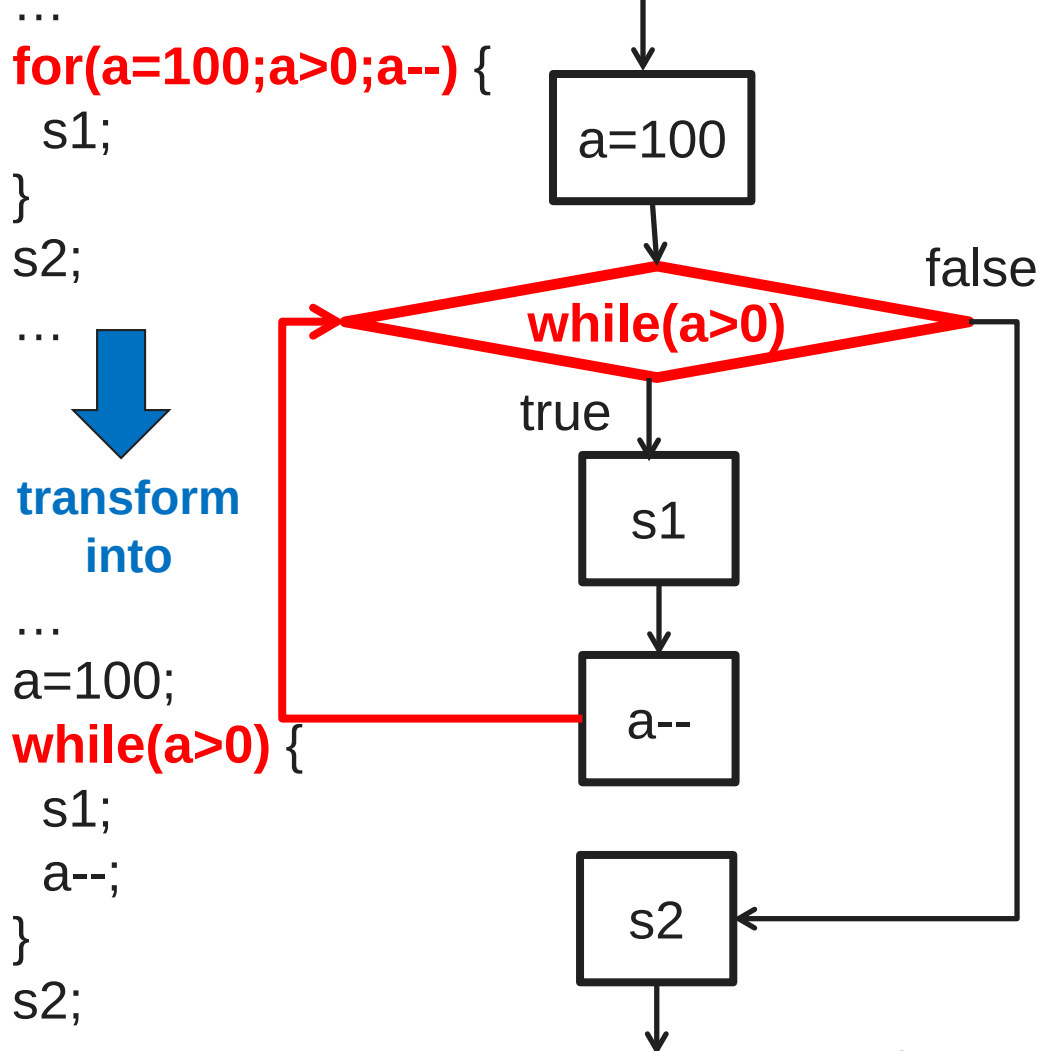
Binary Decision Point – while statement



- ▶ Control flow can change at the decision point
- ▶ One control flow edge into the decision point
- ▶ **Two** control flow edges out of the decision point
 - ▶ True branch
 - ▶ False branch
- ▶ **Loop back** to the decision point from the last statement

Binary Decision Point – for statement

Binary Decision Point

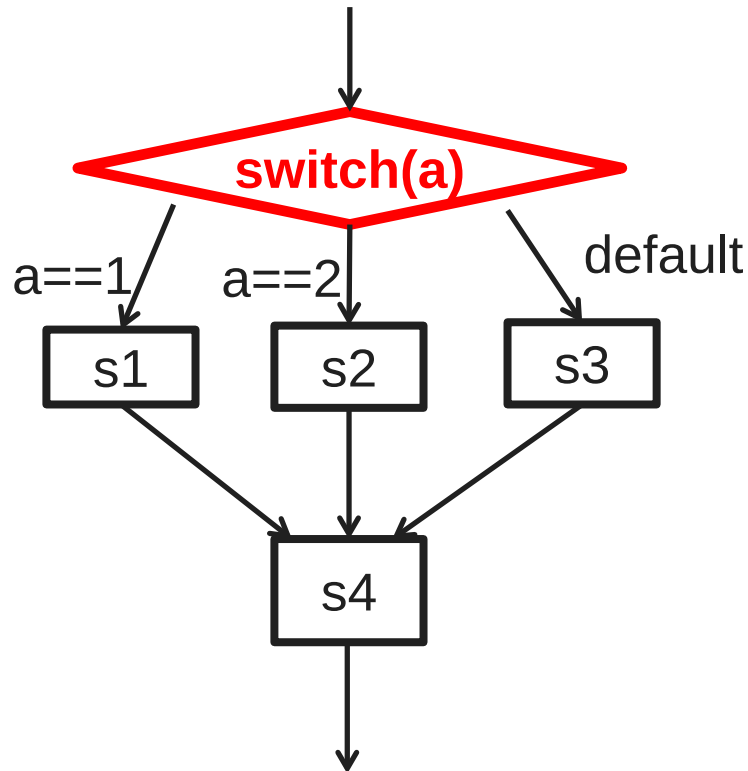


- ▶ Control flow can change at the decision point
- ▶ One control flow edge into the decision point
- ▶ **Two** control flow edges out of the decision point
 - ▶ True branch
 - ▶ False branch
- ▶ **Loop back** to the decision point from the last statement

N-ary Decision Point – switch statement

N-ary Decision Point

```
...  
switch(a) {  
  case 1:  
    s1;  
    break;  
  case 2:  
    s2;  
    break;  
  default:  
    s3;  
}  
s4;  
...
```

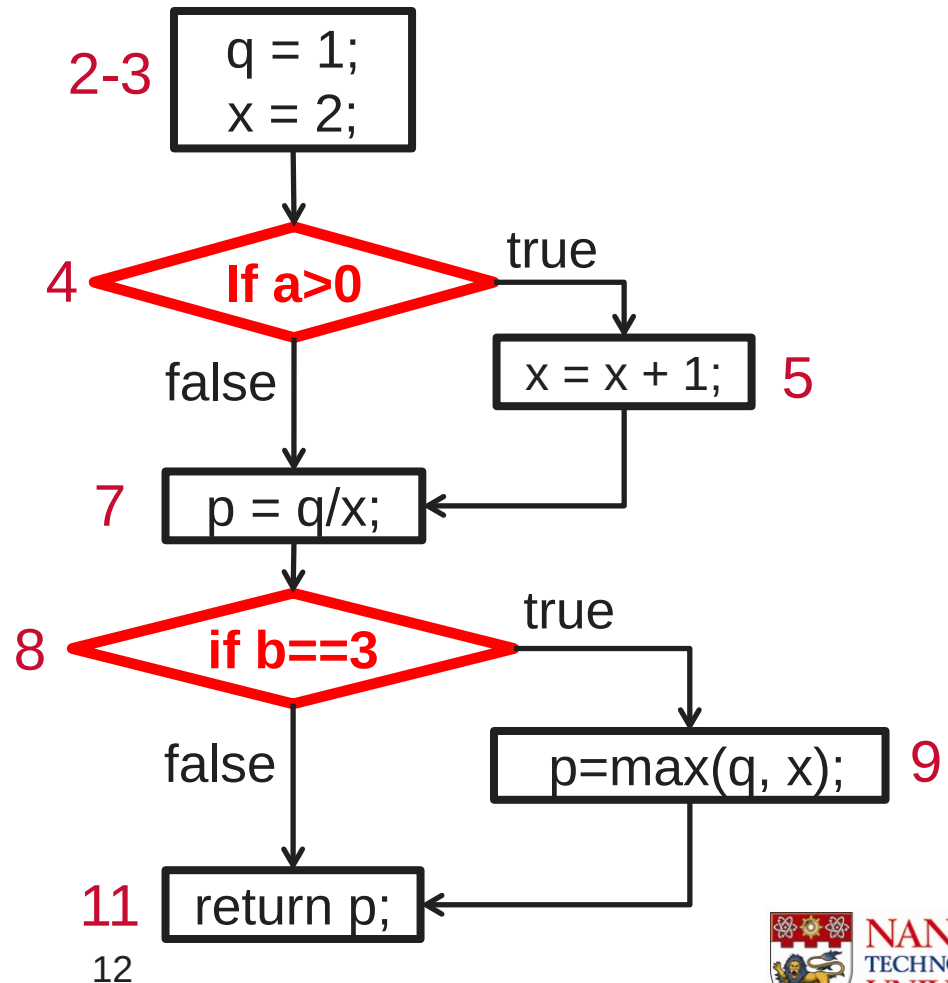


- ▶ Control flow can change at the decision point
- ▶ One control flow edge into the decision point
- ▶ **N** control flow edges out of the decision point

Control Flow Graph - Example

```
int computeP(int a, int b) {
```

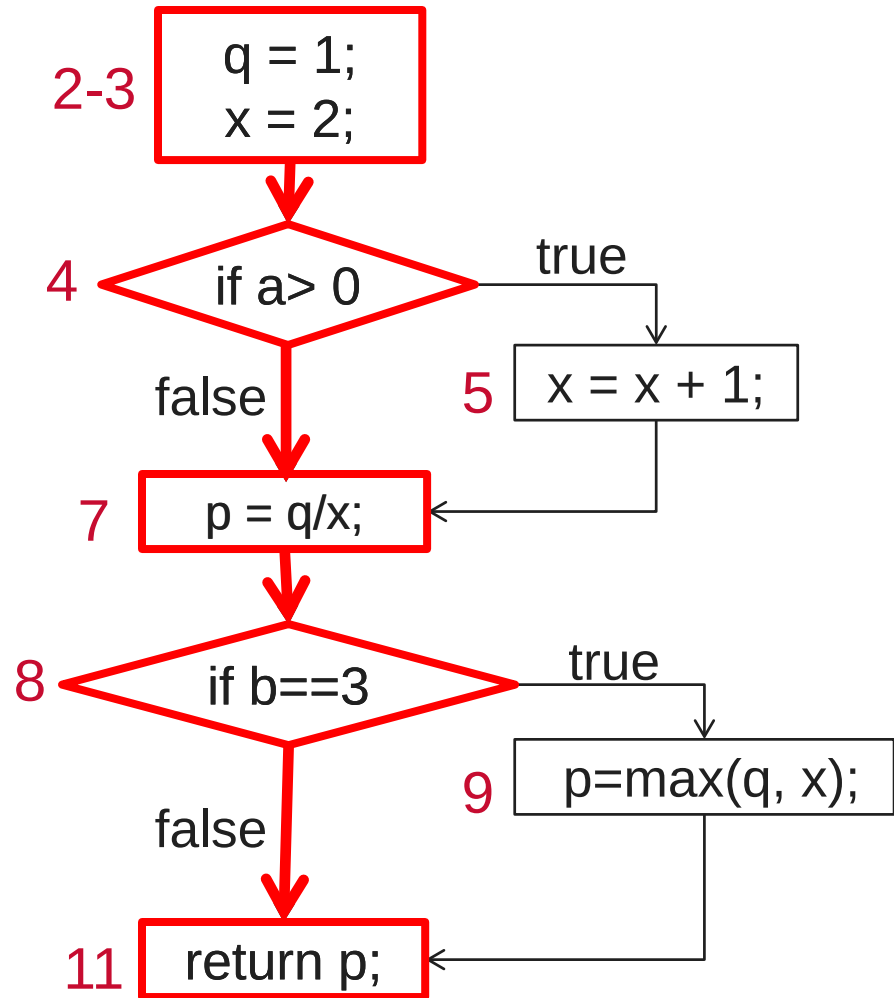
```
1   int q, x, p;  
2   q = 1;  
3   x = 2;  
4   if(a > 0) {  
5       x = x + 1;  
6   }  
7   p = q/x;  
8   if(b == 3) {  
9       p = max(q, x);  
10  }  
11  return p;  
12 }
```



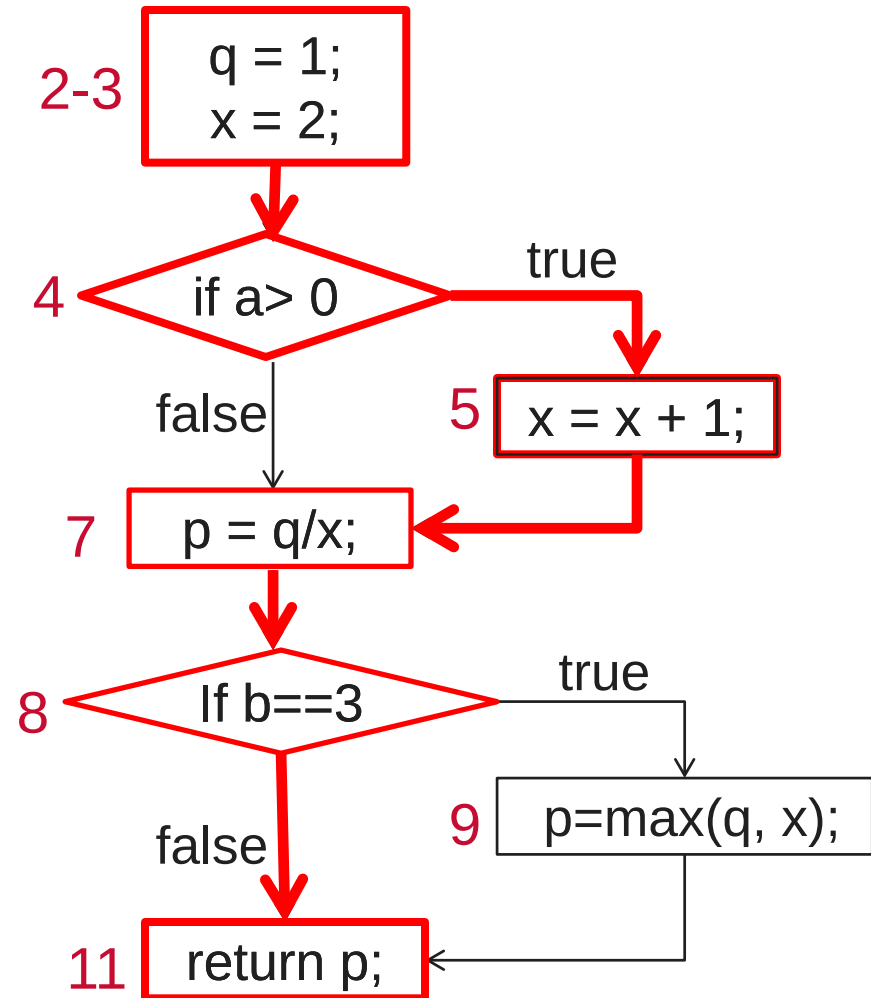
Execution Path Through the CFG

- ▶ A sequence of process blocks and decision points through the CFG of the program

Execution Path – computeP(int,int)



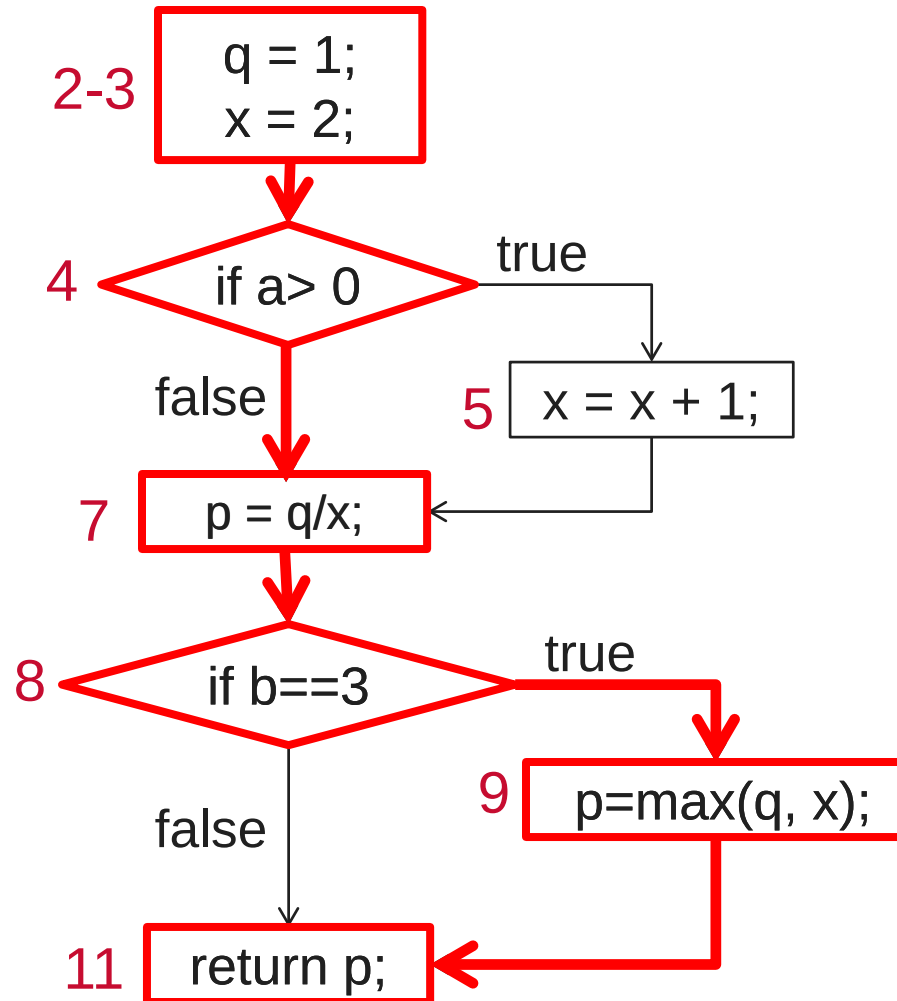
Path#1: 2-3, 4, 7, 8, 11



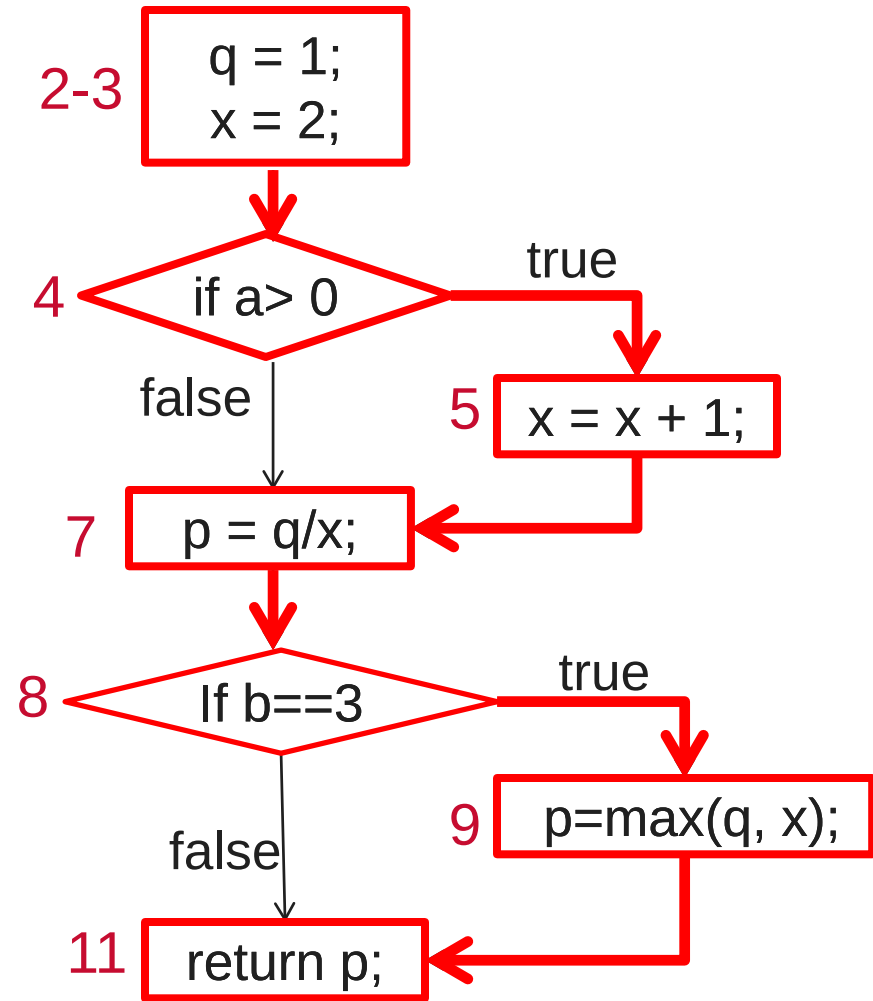
Path#2: 2-3, 4, 5, 7, 8, 11

Other execution paths for computeP(int,int)?

Execution Path – computeP(int,int)



Path#3: 2-3, 4, 7, 8, 9, 11



Path#4: 2-3, 4, 5, 7, 8, 9, 11

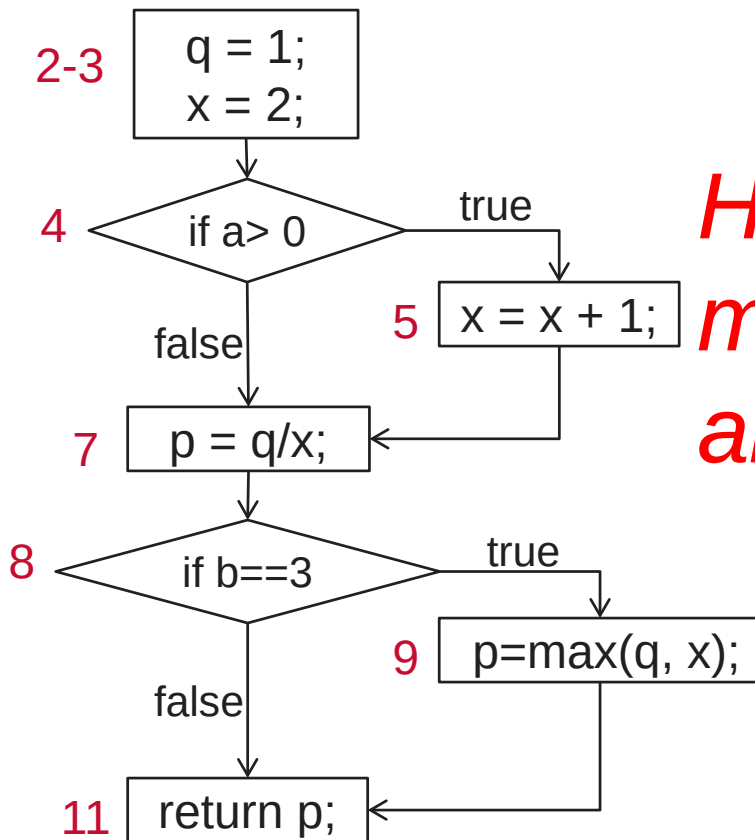
Two decision points → Four execution paths ($2^2 = 4$)

Level 1 - 100% Statement Coverage

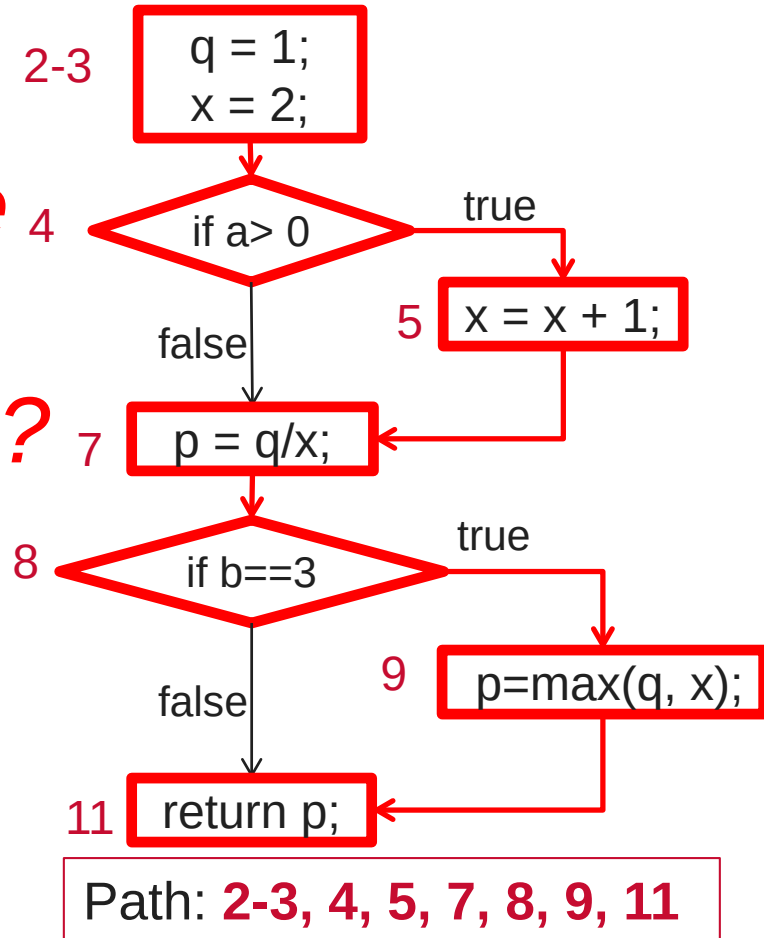
- ▶ Create test cases such that all statements are executed at least once under test
- ▶ Select execution path(s) to cover all the CFG nodes at least once

Level 1 Coverage – computeP(a, b)

- ▶ Select execution path(s) to cover all the **CFG nodes** at least once



Have we missed anything?



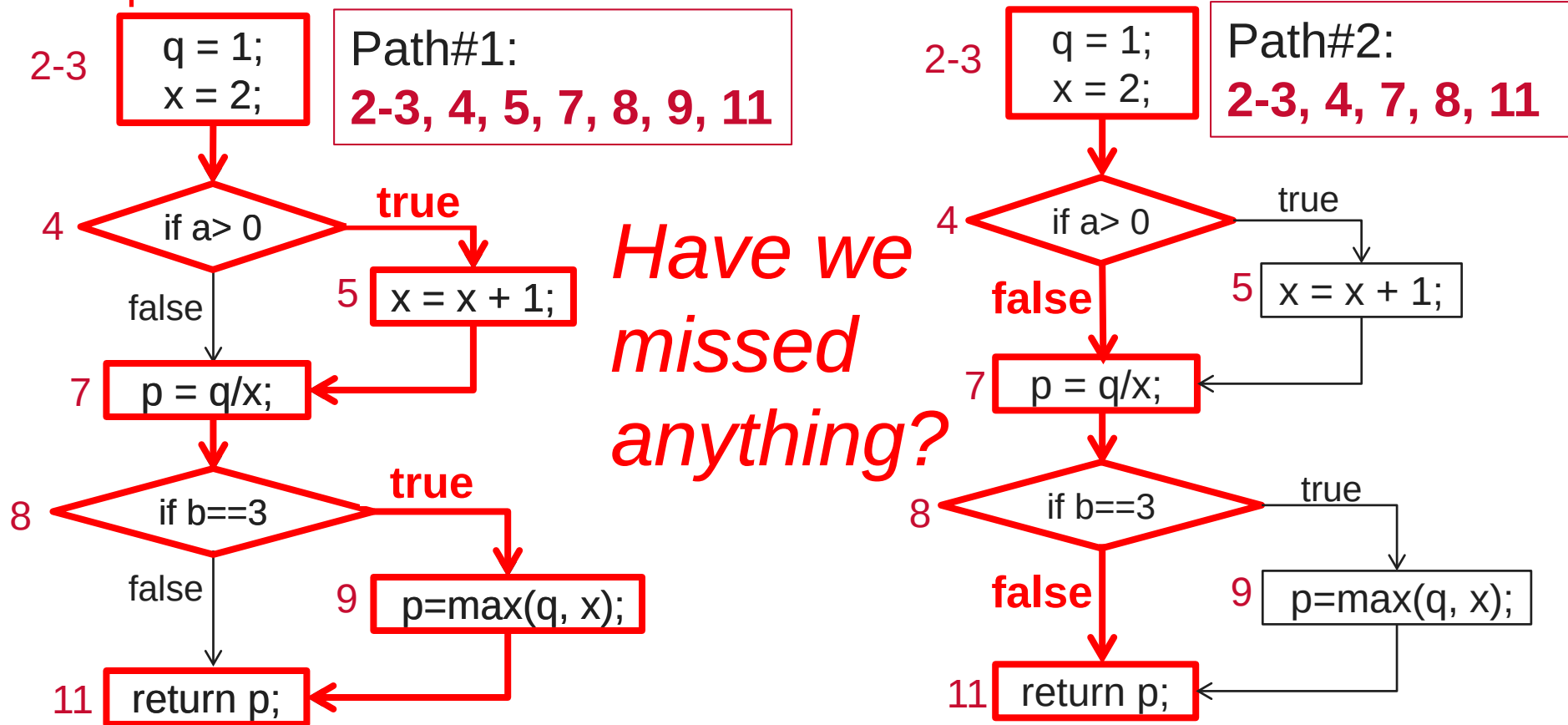
- ▶ Choose input values to execute the selected path
e.g., a=1 and b=3, or a=10 and b=3, etc

Level 2 – 100% Branch Coverage

- ▶ Create test cases such that all branches out of the decision points are executed at least once under test
- ▶ Binary decision point evaluates to TRUE and FALSE outcome at least once

Level 2 Coverage – computeP(a, b)

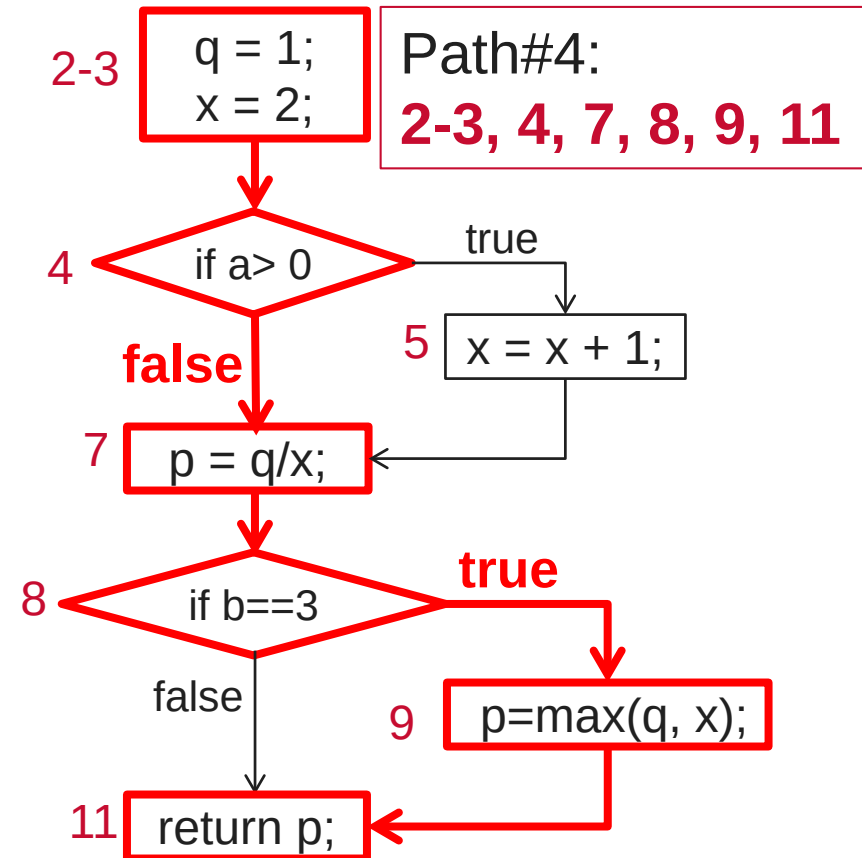
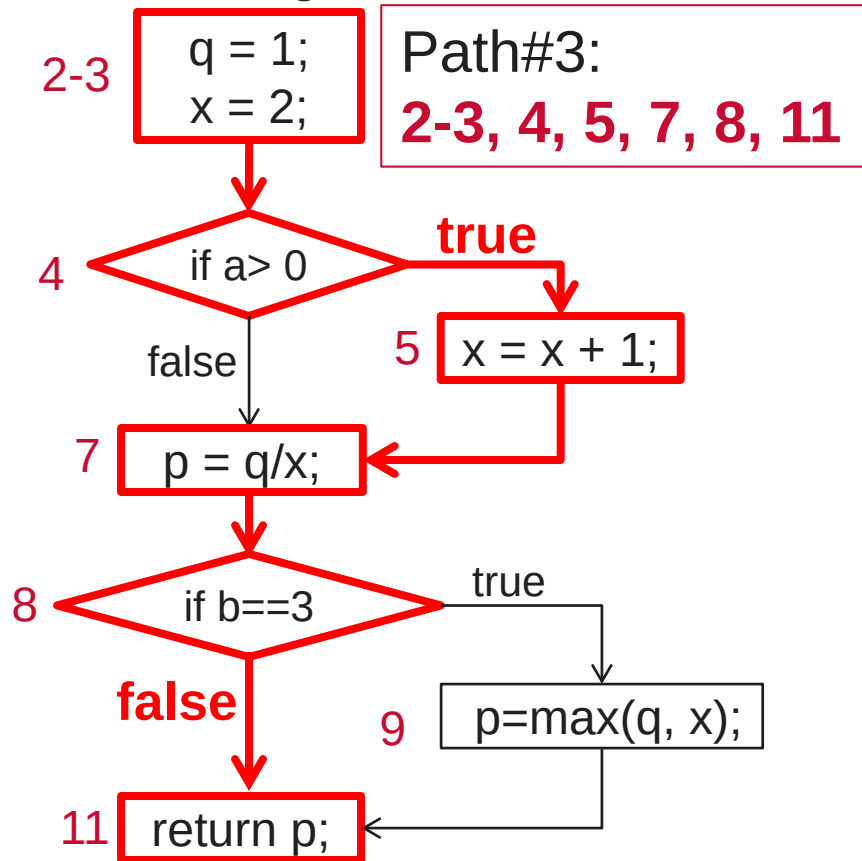
- ▶ Select execution paths to execute all branches of the decision points at least once



- ▶ Choose input values to execute the selected path
e.g., a=1 (true), b=3 (true); and a=0 (false), b=2 (false)

Level 4 – 100% Path Coverage

- Two more execution paths (from level 2) to achieve 100% path coverage of all branches of the decision points at least once



- Choose input values to execute the selected path
e.g., $a=1$ (true), $b=2$ (false); and $a=0$ (false), $b=3$ (true)

Level 4 – 100% Path Coverage

- ▶ Create test cases such that **all execution paths are executed at least once** under test
- ▶ May **not** be **feasible** in general (*we shall see example later*)
 - ▶ Every **decision point** doubles the number of paths (i.e., $2^{|decision|}$)
 - ▶ Every **loop** multiplies the paths by the number of iterations

Level 3 – Basis Path Coverage

- ▶ Create test cases such that the **minimum number of basis paths are executed at least once** under test.
- ▶ What is basis path (a.k.a. Linear Independent Path)?
 - ▶ Each basis path traverses **at least one new control flow edge (i.e. at least one new node)** that **has not** been traversed before the path is defined.
 - ▶ A set of basis paths (or basis set) is a set of linearly independent paths.
 - ▶ The basis set is not unique (*we shall visit this point later*).

Basis Path testing is an effective mechanism to deal with exponential combination of branches.

Basis Path Testing

From Test Fundamentals lecture slide#11

- ▶ **Basis Path Testing** (or structured testing) is a white box testing method used for designing test cases intended to examine all possible paths of execution in a program at least once.
 - ▶ Analyzes **Control flow graph** (CFG)
 - ▶ Determines **Cyclomatic Complexity** (CC) value
 - ▶ Obtains **linearly independent paths**
 - ▶ Generates **test cases** for each path

Basis Path Testing Process

► The tester

1. Construct the **control flow graph (CFG)** of the SUT (System Under Test)
2. Identifies **execution paths** through the CFG of the SUT
 - 2.1 computes the CFG's **Cyclomatic Complexity (CC)**
 - 2.2 selects a set of **#CC basis paths**
3. ...

The rest of steps are the same as the slide #3
4. ...
5. ...

► The test engine

1. ...
2. ...
3. ...

***Basis Path testing guarantees
100% statement and branch
coverage***

Cyclomatic Complexity of a CFG

- ▶ CC measures how complex your program is in terms of the number of decision points (i.e., if/while/for/switch).
 - ▶ The more decision points (i.e., if/while/for/switch) a program has → The higher the CC → The more basis paths you need to test.
- E.g. if your program has CC value 6, then you've to identify 6 basic paths to test.

Cyclomatic Complexity of a CFG

- ▶ Use the following two equations to compute CC
 - ▶ $CC = |\text{edges}| - |\text{nodes}| + 2$
 - ▶ $CC = |\text{decisionpoint}| + 1$, if all decision points are binary, i.e., one true branch + one false branch

The two equations compute the same value if all decision points are binary

Cyclomatic Complexity – Example

```
int computeP(int a, int b) {
```

```
1  int q, x, p;
```

```
2  q = 1;
```

```
3  x = 2;
```

```
4  if(a > 0) {
```

```
5      x = x + 1;
```

```
6  }
```

```
7  p = q/x;
```

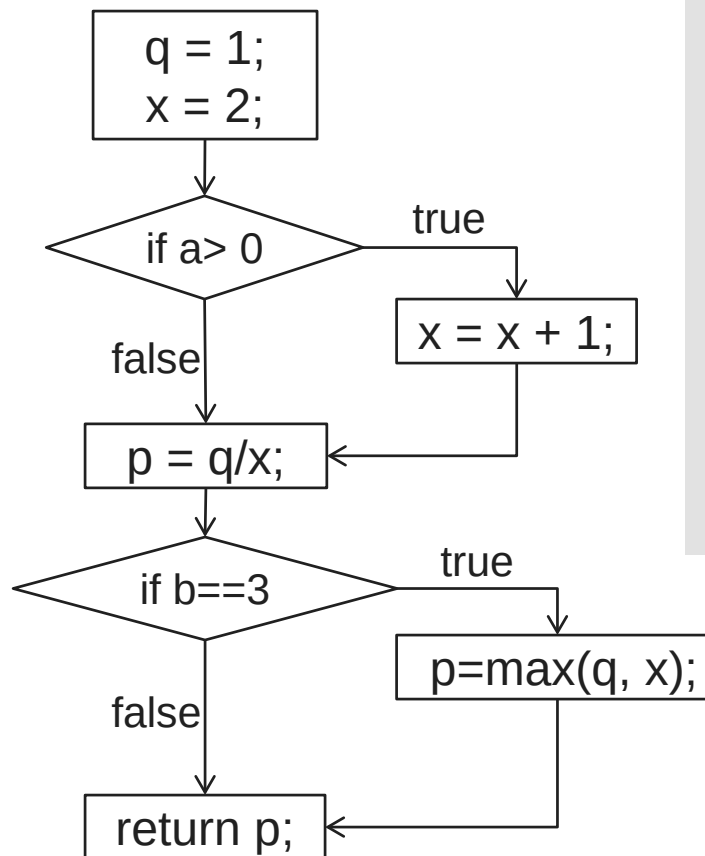
```
8  if(b == 3) {
```

```
9      p = max(q, x);
```

```
10 }
```

```
11 return p;
```

```
}
```



$|edges| = 8$

$|nodes| = 7$

$$CC = 8 - 7 + 2 = 3$$

Or,

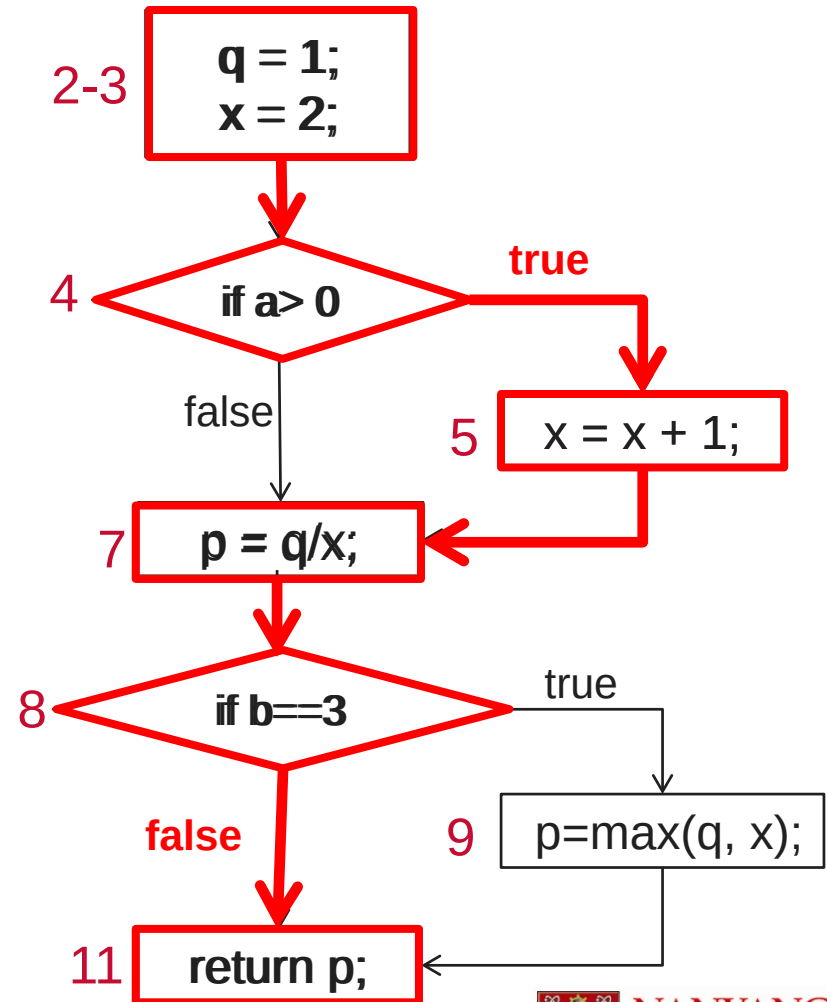
$|decisionpoint| = 2$

$$CC = 2 + 1 = 3$$

Select a Set of Basis Paths

- ▶ Pick a “**baseline**” path
- ▶ Reasonably “typical” path of execution
- ▶ Most important path from the tester’s view

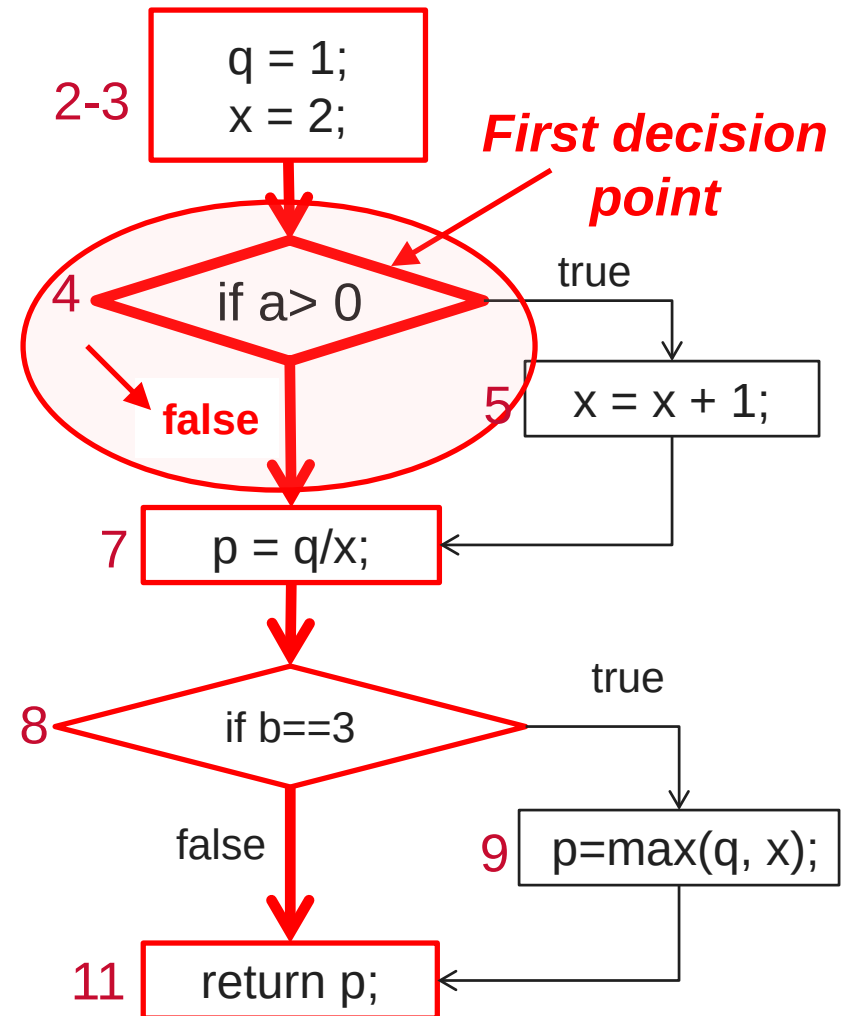
Basis path#1 (baseline):
2-3, 4, 5, 7, 8, 11



Select a Set of Basis Paths

- ▶ Change the outcome of the **first** decision point
- ▶ Keep the maximum number of **other** decision points the same

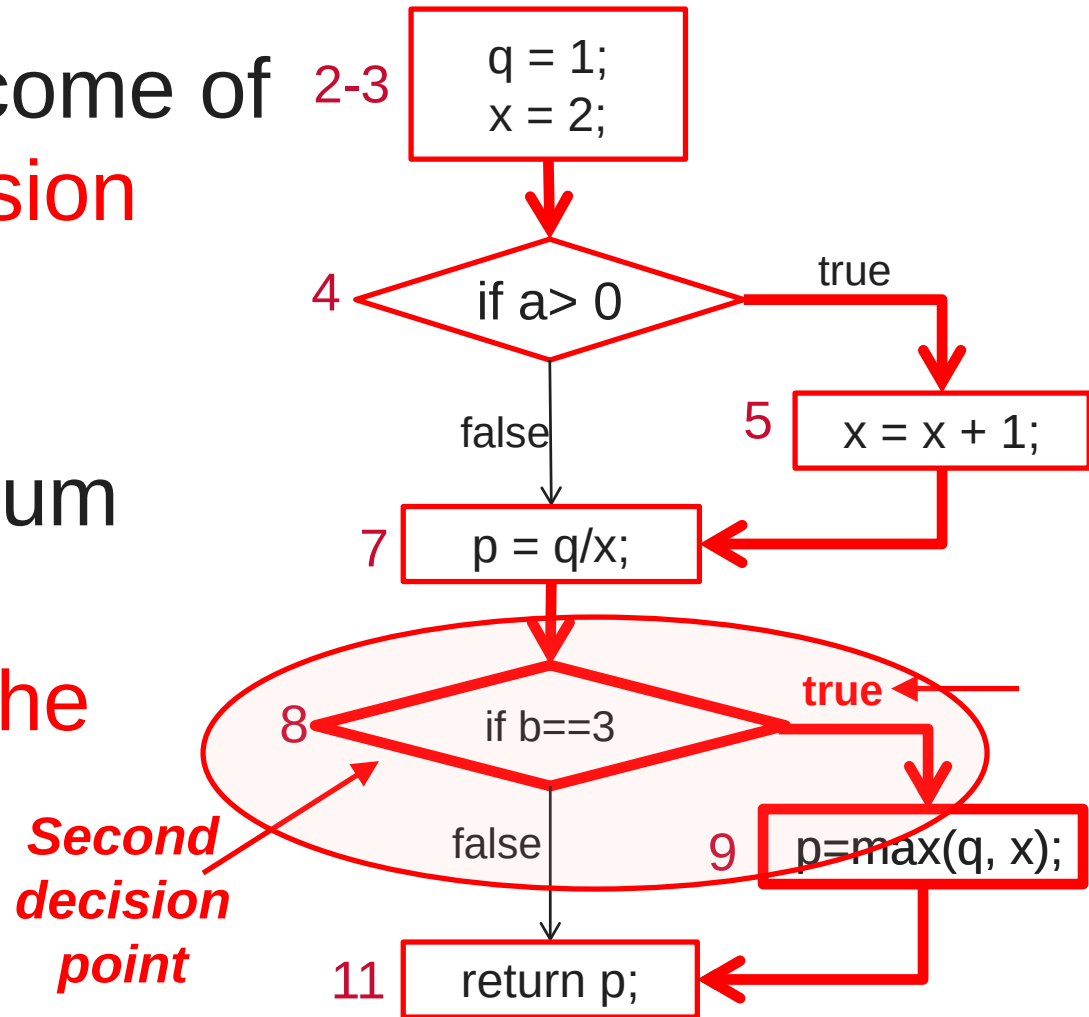
Basis path#2:
2-3, 4, 7, 8, 11



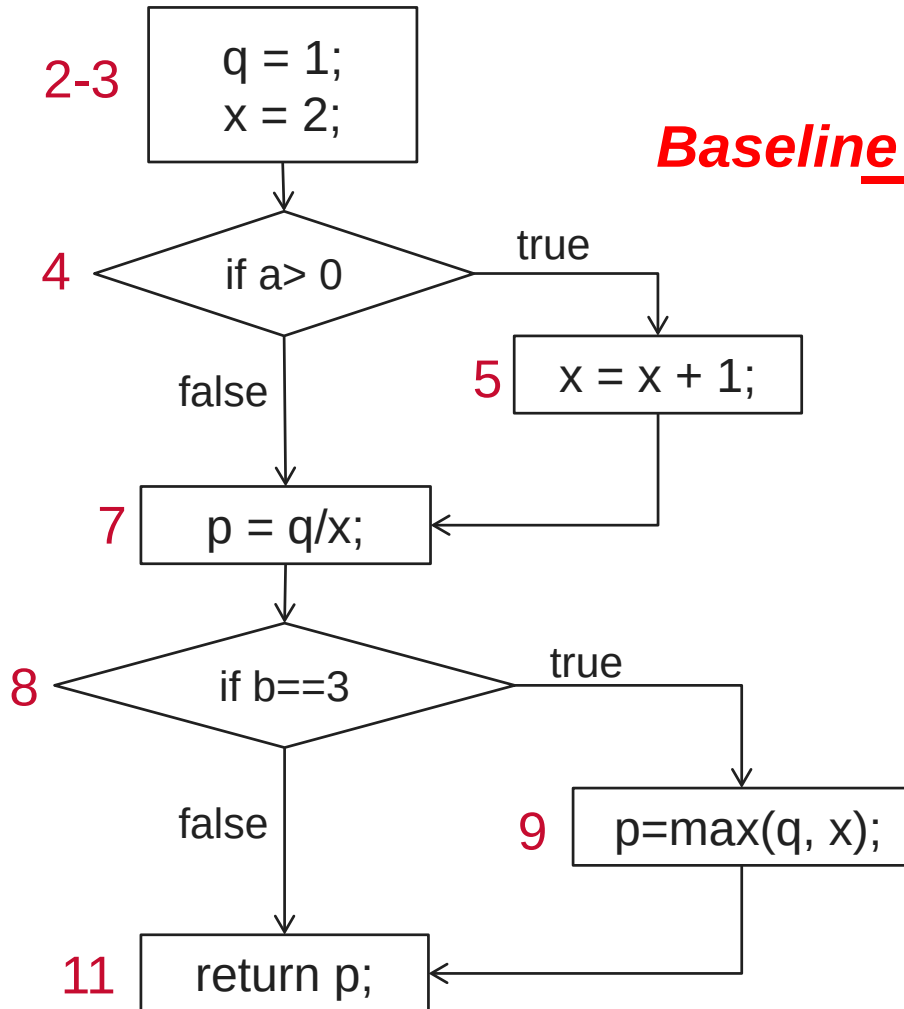
Select a Set of Basis Paths

- ▶ **Change** the outcome of the second decision point
- ▶ **Keep** the maximum number of **other** decision points the same

Basis path#3 :
2-3, 4, 5, 7, 8, 9, 11



Create a Test Case for Each Basis Path



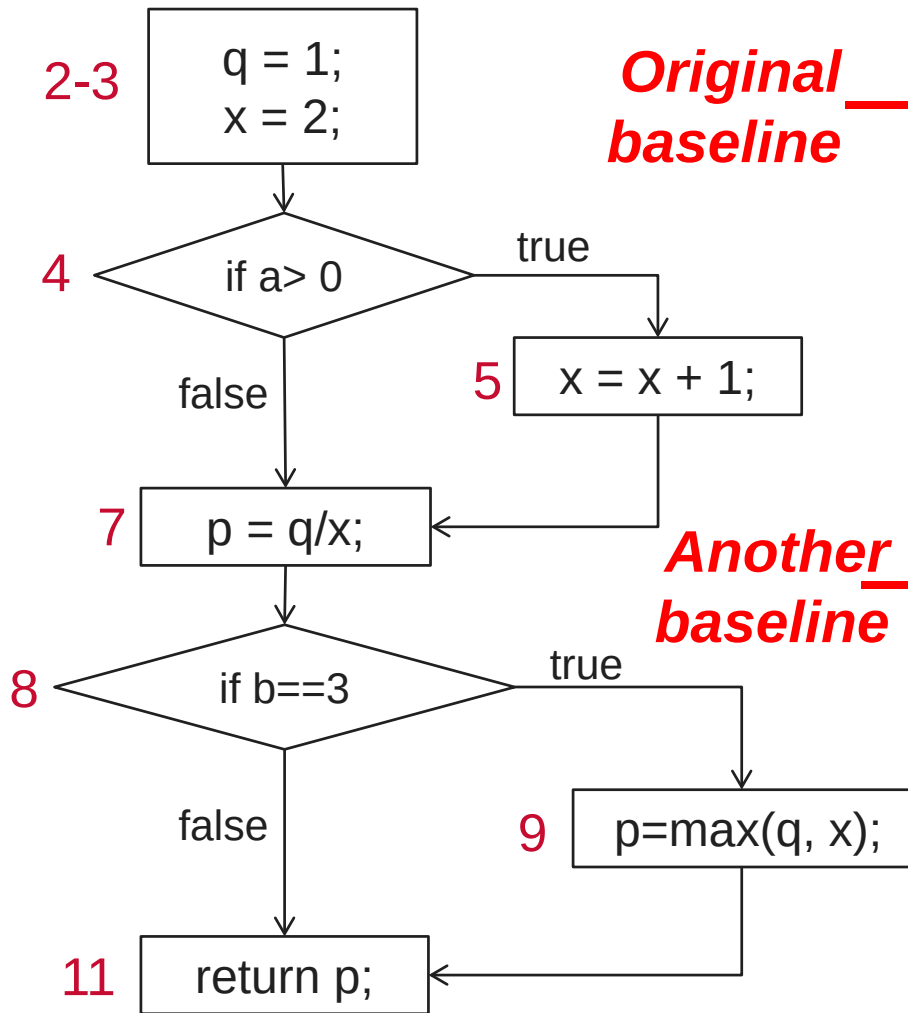
► One set of basis paths

- I. 2-3, 4, 5, 7, 8, 11
- II. 2-3, 4, 7, 8, 11
- III. 2-3, 4, 5, 7, 8, 9, 11

► Three test cases

- I. $a = 4$; $b = 2$
- II. $a = 0$; $b = 5$
- III. $a = 7$; $b = 3$

Basis Paths for computeP()



► One set of basis paths

I. 2-3, 4, 5, 7, 8, 11

II. 2-3, 4, 7, 8, 11

III. 2-3, 4, 5, 7, 8, 9, 11

► Another set of basis paths

I. 2-3, 4, 7, 8, 11

II. 2-3, 4, 5, 7, 8, 11

III. 2-3, 4, 7, 8, 9, 11

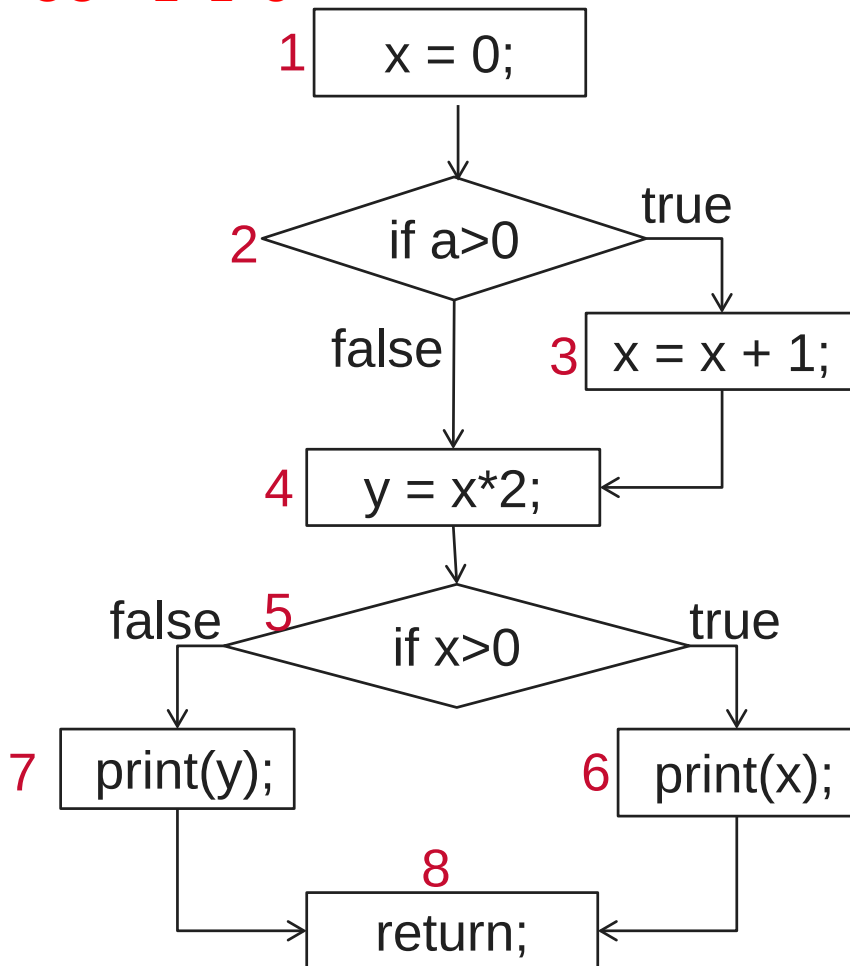
*The **third** paths in the two sets are different*

The basis set is not unique; but each basis set has the same number of basis paths (i.e. 3 in this case)

Not All Basis Paths are Feasible

Consider another program logic

CC = 2+1=3



► One set of basis paths

I. 1, 2, 3, 4, 5, 6, 8 ← **Baseline**

II. 1, 2, 4, 5, 6, 8 (infeasible)

III. 1, 2, 3, 4, 5, 7, 8 (infeasible)

- **Two** infeasible paths

► One test case

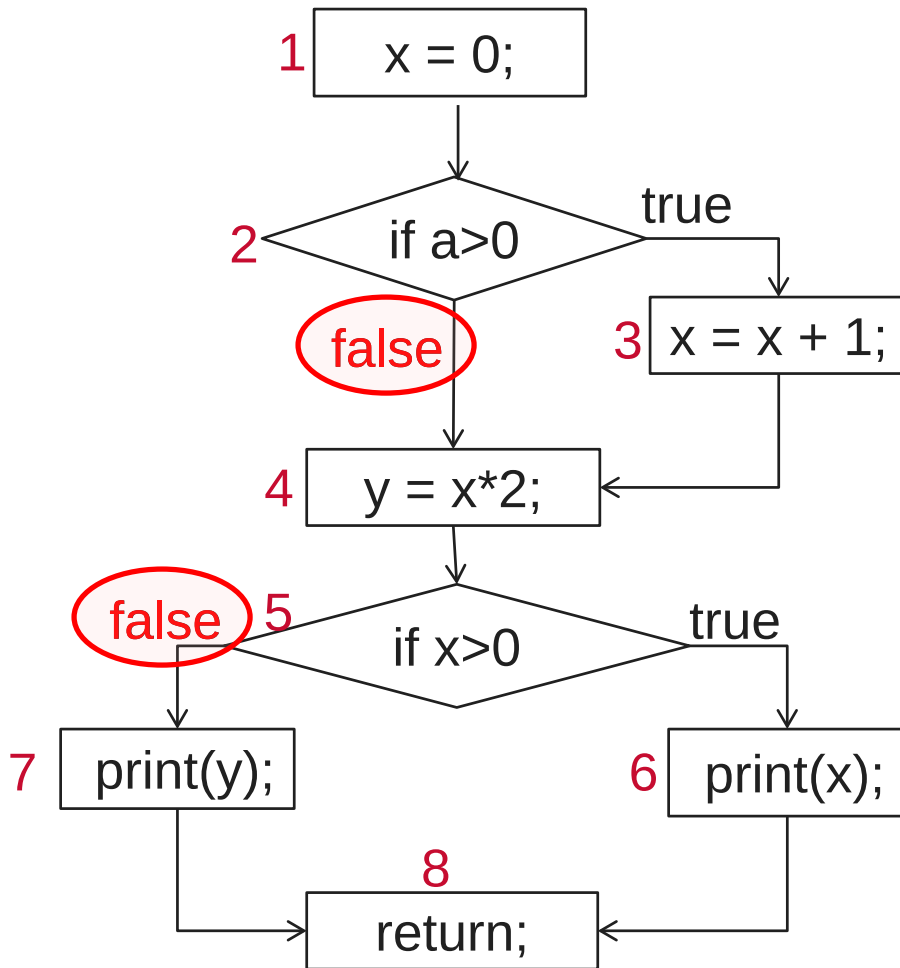
I. a = 4

II. Infeasible basis path

III. Infeasible basis path

Fail to test 2, 4, 5, 6, 8, and 5, 7, 8, branches

Minimize Infeasible Basis Paths



► Another set of basis paths

I. 1, 2, 3, 4, 5, 6, 8

II. ~~1, 2, 4, 5, 6, 8~~ (infeasible)

III. 1, 2, 4, 5, 7, 8 (change **both decision points** at the same time)

- *One infeasible path*

► Two test case

I. `a = 4`

II. Infeasible basis path

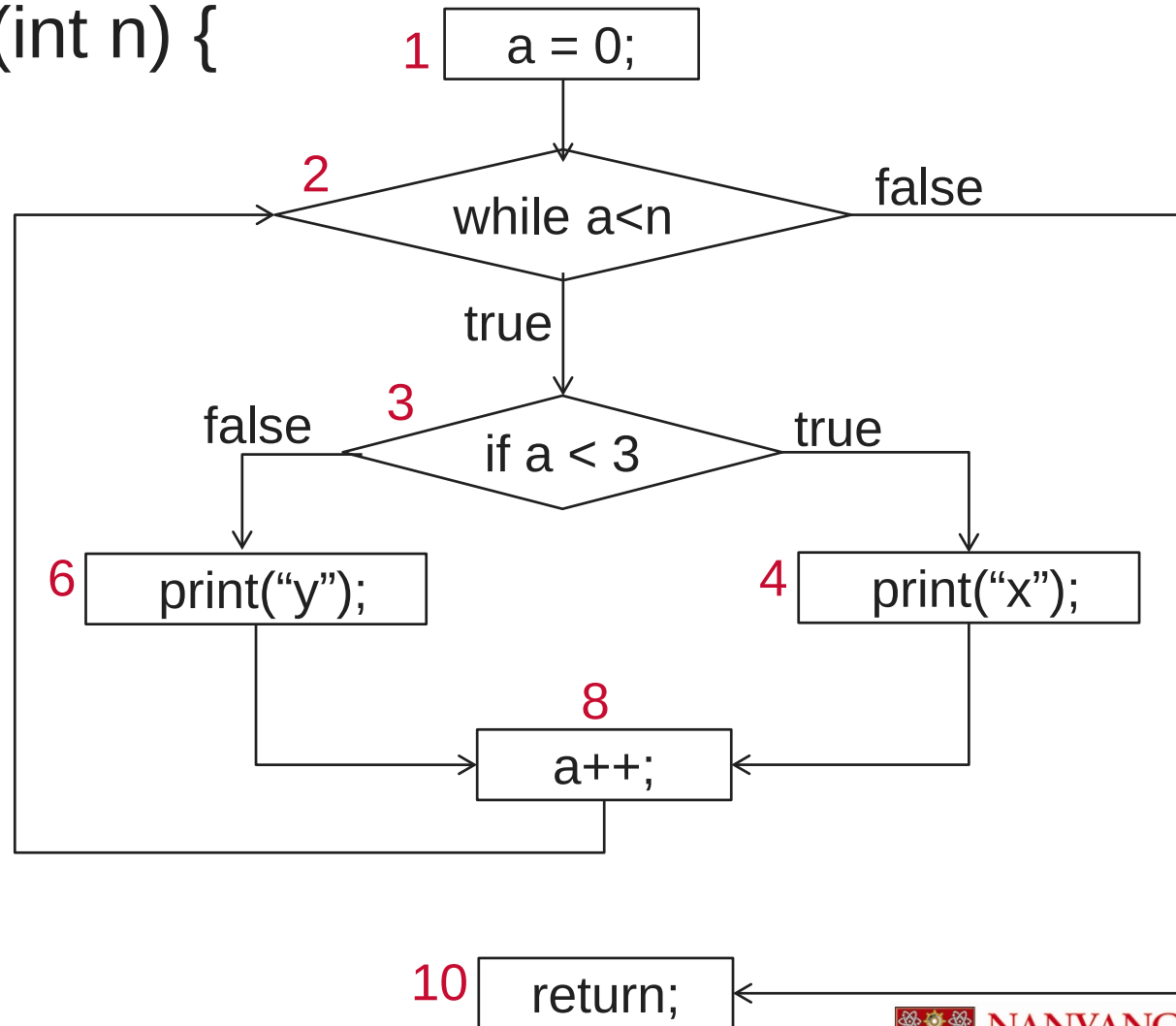
III. `a = 0`

How to Deal with Loop?

- ▶ When selecting basis paths, select the loop **only zero and once** (no need to consider iteration)
- ▶ When **selecting baseline paths**, select **false branch first** at loop decision point, i.e., ***do not enter the loop***
- ▶ When choosing input values to execute the path, the real execution path may execute loop several times (*see example next slide*).

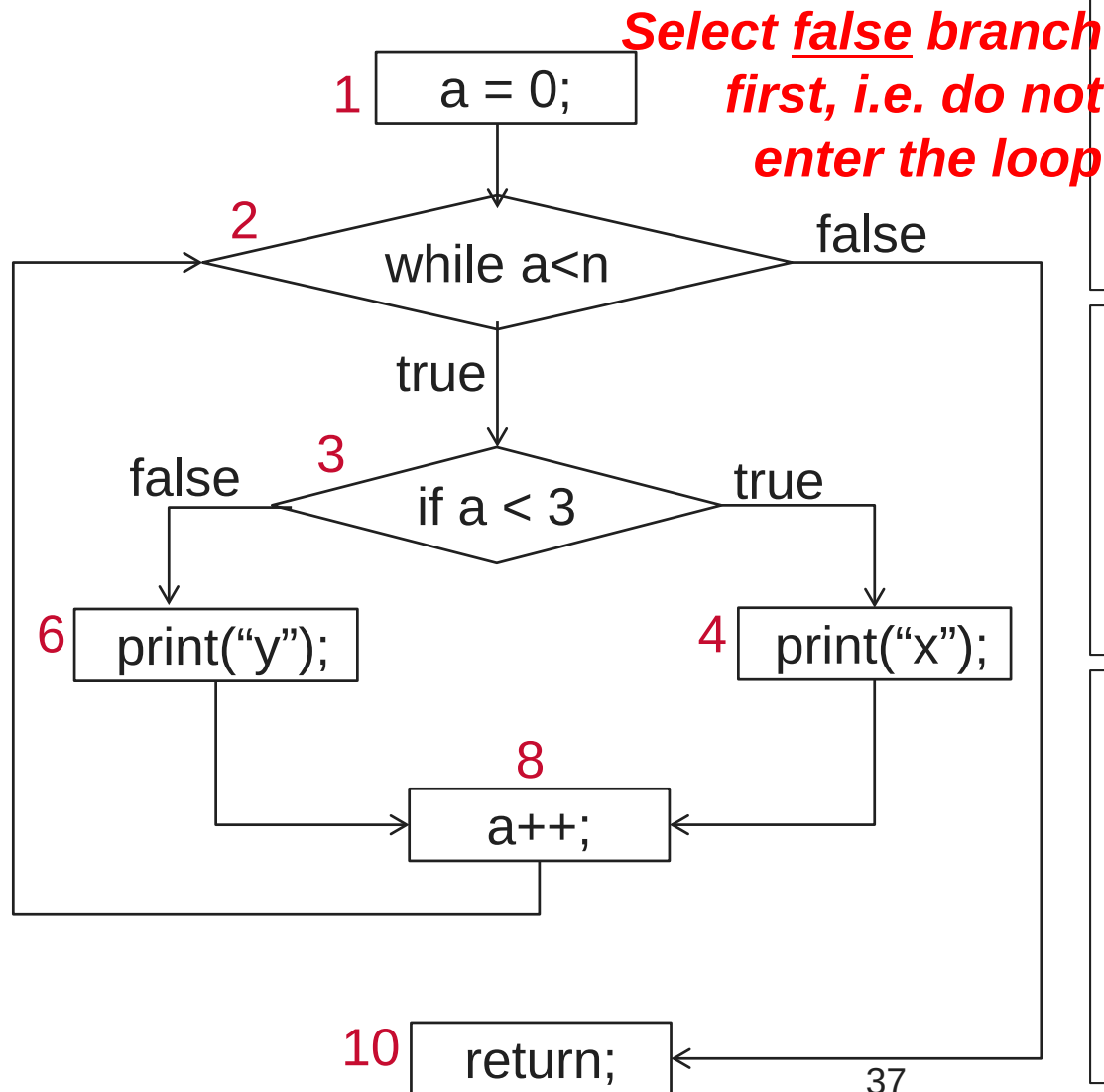
Exercise – Basis Path Testing

```
doSomething(int n) {  
1  int a = 0;  
2  while(a < n) {  
3    if(a < 3) {  
4      print("X");  
5    } else {  
6      print("Y");  
7    }  
8    a++;  
9  }  
10 return;  
11 }
```



Exercise – Basis Path Testing

$$CC = 2 + 1 = 3$$



► Three basis paths

I. 1, 2, 10 ← **Baseline**

II. 1, 2, 3, 4, 8, 2, 10

III. 1, 2, 3, 6, 8, 2, 10

► Three test cases

I. n = 0

II. n = 1

III. n = 4

► Real execution paths

I. 1, 2, 10

II. 1, 2, 3, 4, 8, 2, 10

III. 1, 2, 3, 4, 8, 2, 3, 4, 8, 2, 3, 4, 8, 2, 3, 6, 8, 2, 10

Testing – Both Testings are Important!

▶ Black Box Testing

- ▶ Equivalence class testing (range of values, discrete values)
- ▶ Boundary value testing (only applicable to range of values)

▶ White Box Testing

- ▶ Control flow graph (CFG)
- ▶ Statement, branch, basis path, path coverage
- ▶ Basis path testing
 - ▶ Cyclomatic complexity (how many basis paths you need to test)
 - ▶ Select basis paths